AsyncAir

Trabajo 2

(JavaScript Asincrónico)

Magdalena Cerón
Pamela Hernández
Vicente Retamal
Sofía Retamales

Solución

```
├ check-in.html
├ checkIn.js
├ boardingPass.js
└ Styles
    ├ img
    └ style.css
```

Función validarPasaporte(id)

```
const validarPasaporte = (id) => {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (id % 2 == 0) {
        reject(new Error("El ID de pasaporte es inválido"));
      }
      resolve("Id válido");
    }, 1500);
  });
};
```

Falla si el ID es número par.

Tarda 1.5 segundos.

Función verificarRestriccionesVisa(id)

```
const verificarRestriccionesVisa = (id) => {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      return Math.random() < 0.3
        ? reject(new Error("Visa no válida para el
destino."))
        : resolve("Visa verificada");
    }, 2000);
  });
};
```

Falla de forma aleatoria el 30% de las veces.

Tarda 2 segundos.

Función asignarAsiento()

```
const asignarAsiento = () => {
  const rows = Array.from({ length: 30 }, (_, i) => i + 1);
  const seats = ["A", "B", "C", "D", "E", "F"];

  return new Promise((resolve) => {
    setTimeout(() => {
      const row = rows[Math.floor(Math.random() *
rows.length)];
      const seat = seats[Math.floor(Math.random() *
seats.length)];
      resolve(`${row}${seat}`);
    }, 1000);
  });
};
```

Elige una fila al azar entre 1 y 30.

Elige una letra al azar entre A y F.

Tarda 1 segundo.

Función generarPaseAbordar(datos)

```
const generarPaseAbordar = (datos) => {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(datos);
    }, 500);
  });
};
```

Retorna los datos obtenidos anteriormente.

Tarda 0.5 segundos.

Función flujoCheckIn(id, log)

```
const flujoCheckIn = (pasajeroId, log) => {
  const validationPromises = [validarPasaporte(pasajeroId).then((msg) => log(msg)),
    verificarRestriccionesVisa(pasajeroId).then((msg) => log(msg))];

  return Promise.all(validationPromises)
    .then(() => {
      log(`Asignando asiento...`);
      return asignarAsiento();
    })

    .then((asiento) => {
      console.log(asiento);

      return generarPaseAbordar({
        pasajeroId,
        asiento
      });
    })
  };
```

Se validan la visa y el id del pasajero de forma concurrente.

Si las válidaciones son exitosas se consulta por el asiento del usuario.

Al finalizar el proceso se genera el pase del usuario con toda la información previa.

Función iniciarCheckIn()

```
const iniciarCheckIn = () => {
  const pasajeroId = parseInt(document.getElementById("input-id").value.trim());
  if (
    pasajeroId === null ||
    pasajeroId === undefined ||
    pasajeroId === "" || isNaN(pasajeroId)) return;

  const logsBox = document.getElementById("logs-box");
  const inputButton = document.getElementById("input-btn");
  inputButton.disabled = true;
  logsBox.innerHTML = "";

  const log = (message) => {
    logsBox.innerHTML += `<p>${message}</p>`;
  }

  log(`ID ingresado: ${pasajeroId}`);
  log(`Iniciando Validaciones...`);
```

Se desactiva el botón para evitar condiciones de carrera.

Función especifica para actualizar el log de eventos.

Función iniciarCheckIn()

```
Promise.race([flujoCheckIn(pasajeroId, log), checkInTimeout()])
  .then((pase) => {
    inputButton.disabled = false;
    mostrarBoardingPass(pase);
  })

  .catch((error) => {
    console.log(error);
    logsBox.innerHTML = `<p>Error: ${error.message}</p>`;
    inputButton.disabled = false;
  });
};
```

Timeout global: Se retorna la primera promesa que termina.

Si todo salió bien, el usuario tiene acceso a su pase de abordar.

AsyncAir

Uso de IA

¿Cómo mostrar tarjeta de embarque de manera funcional?

1

Reemplazar HTML dinámicamente (innerHTML)

Generar el boarding pass como un string HTML desde JavaScript y reemplazar el contenido de la página.

2

Manejo de vistas

Se controla la navegación entre vistas mediante propiedades declarativas que indican qué vista mostrar u ocultar.

3

Uso de <template> HTML

Se utiliza un template HTML que se clona dinámicamente y se rellena con los datos del pasajero.

- **Se define la vista como un objeto**
- **funciones puras generan el estado**
- **Funciones de render aplican cambios al DOM**

¿Cómo mostrar tarjeta de embarque de manera funcional?

2 Manejo de vistas

```
const mostrarBoardingPass = (pase) =>
{
renderBoardingPass(crearVistaBoarding
Pass(pase));
};
```

Función **volverAlCheckIn** ocupa esta misma lógica

```
const renderBoardingPass = (vista) => {
 const checkinView = document.getElementById("checkin-view");
 const boardingView = document.getElementById("boarding-view");
 const template = document.getElementById("boarding-template");
 checkinView.style.display = vista.checkinDisplay;
 boardingView.style.display = vista.boardingDisplay;
 boardingView.querySelectorAll(".boarding-card-frame").forEach((el) =>
el.remove());
 const clone = template.content.cloneNode(true);
 const pasajeroEl = clone.querySelector(".pasajero");
 const asientoEl = clone.querySelector(".asiento");
 if (pasajeroEl) pasajeroEl.textContent = vista.pasajero;
 if (asientoEl) asientoEl.textContent = vista.asiento;
 boardingView.prepend(clone);
};
```

¿Cómo mostrar tarjeta de embarque de manera funcional?

3 Uso de <template> HTML

```
<template id="boarding-template">
  <section class="boarding-card-frame">
                    ...
  </section>
</template>
```

```
const renderBoardingPass = (vista) => {
 const checkinView = document.getElementById("checkin-view");
 const boardingView = document.getElementById("boarding-view");
   const template = document.getElementById("boarding-
template");
 checkinView.style.display = vista.checkinDisplay;
 boardingView.style.display = vista.boardingDisplay;
 boardingView.querySelectorAll(".boarding-card-frame").forEach((el) =>
el.remove());
 const clone = template.content.cloneNode(true);
 const pasajeroEl = clone.querySelector(".pasajero");
 const asientoEl = clone.querySelector(".asiento");
 if (pasajeroEl) pasajeroEl.textContent = vista.pasajero;
 if (asientoEl) asientoEl.textContent = vista.asiento;
 boardingView.prepend(clone);
};
```

Reflexión

Reflexión

- Trabajo pequeño → difícil dividir tareas de forma completamente independiente.
- Alta dependencia entre funciones → obliga a coordinar constantemente avances.
- Comunicación continua fue clave para evitar bloqueos.
- La IA es útil para conocer las distintas opciones de implementación disponibles, pero la decisión final recae en nosotros.

Gracias!